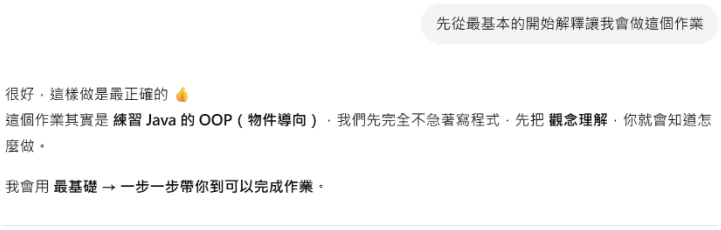
H1 Report D1249433 吳俊毅資訊三丙

設計方法概述

要設計之前我需要先搞清楚總共有那些東西再去下手，我發現這整個遊戲就是棋子、棋盤、還有控制整個遊戲的主程式，**main** 程式控制棋盤 棋盤再管理棋子，然後棋子會有它的屬性，因為很久沒有用 **java** 寫程式了所以這次我全部交給 **ai** 先幫我成出來，但我不是但純叫他寫出來而已，我是叫他一步一步從遇到問題時要從哪裡下手到出現 **bug** 時可能出現的原因全部教會我，因此花了不少時間。



他教我先找名詞 像是棋子 玩家 棋盤 這些都是 **class** 然後他們會有各自的屬性 像是棋會有 **name**、**weight**、**side**、**loc** 等屬性，然後我們先把程式大綱出來，從這遊戲的幾個動作下手，先 **1** 建立棋子 **2** 印棋子 **3** 棋盤 **4** 移動，架構出來之後再來開始寫細節的問題，先從 **chess** 開始寫，把資料都寫好之後開始寫控制棋子的 **ChessGame**，然後 **main** 程式負責遊戲流程 **1** 建立 **ChessGame**

2generateChess() 3generateChess() 4 隨機生成棋子 然後遊戲主迴圈每次都會顯示棋盤 然後輸入玩家位置接著判斷 判斷是看棋子有沒有翻開 有翻開就顯示輸入的位置的棋，並且加入一些規則像是距離只能間隔 **1**。

```
  A B C D E F G H
1 卒 X X X X X X 士
2 卒 X X X X X X 兵
3 炮 相 象 X X X X X
4 兵 X 車 X X X X X
choose position: H1
move to: H2
invalid move
```

你的程式其實 **運作是正確的**，`invalid move` 出現是因為 **規則判斷不通過**。
我們一步一步看你這個畫面發生了什麼。

我先從簡單的開始做一開始沒有分大小，只要是棋子都可以互吃，結果我發現出現 **invalid move**，於是我在 **move()**裡加入一行 **System.out.println("target open: " + c2.open);**這樣當我吃棋子的時候就會印出來 **target open: true**

或是 **target open: false** 但結果還是 **invalid move** 所以就是其他問題我發現我寫這樣 **String side = index < 16 ? "black" : "red";**

但我是先 **shuffle** 再用 **index** 判斷，就是棋子順序被打亂
但 **side** 還是用 **index** 決定，所以才會出錯，所以後面我直接用棋子名稱判斷陣營

```java
List<String> list = new ArrayList<>(Arrays.asList(pieces));

Collections.shuffle(list);

int index = 0;

for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 8; j++) {

        String name = list.get(index);

        String side = getSide(name);

        int weight = getWeight(name);

        board[i][j] = new Chess(name, side, weight);

        index++;
    }
}
```

先將所有棋子放入 List 中，使用 Collections.shuffle() 隨機打亂順序，

再透過雙層迴圈將棋子依序放入 4x8 的棋盤中，確保每次遊戲開始時棋子排列都不同。

```java
public boolean canEat(Chess a, Chess b) {  1 usage  new *

    String A = a.name;
    String B = b.name;

    if ((A.equals("兵") || A.equals("卒")) &&
            (B.equals("將") || B.equals("帥")))
        return true;

    if ((A.equals("將") || A.equals("帥")) &&
            (B.equals("兵") || B.equals("卒")))
        return false;

    if ((A.equals("兵") || A.equals("卒")) &&
            B.equals("炮"))
        return false;

    return a.weight >= b.weight;
}
```

這是規則判斷 兵 / 卒 可以吃 將 / 帥 ，兵 / 卒 可以吃 將 / 帥，兵 / 卒 可以吃 將 / 帥

```java
int dist = Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);

if (dist != 1)
    return false;
```

這段程式用來判斷棋子是否移動到 相鄰位置

我把程式 bug 找出來修改好之後才加上完整的規則

```
choose position: a1
  A B C D E F G H
1 將 X X X X X X X
2 X X X X X X X X
3 X X X X X X X X
4 X X X X X X X X
choose position: a2
  A B C D E F G H
1 將 X X X X X X X
2 士 X X X X X X X
3 X X X X X X X X
4 X X X X X X X X
choose position: a3
  A B C D E F G H
1 將 X X X X X X X
2 士 X X X X X X X
3 象 X X X X X X X
4 X X X X X X X X
choose position: a4
  A B C D E F G H
1 將 X X X X X X X
2 士 X X X X X X X
3 象 X X X X X X X
4 仕 X X X X X X X
choose position: a4
move to: a3
  A B C D E F G H
1 將 X X X X X X X
2 士 X X X X X X X
3 仕 X X X X X X X
4 . X X X X X X X
```

```
choose position: a3
move to: a2
  A B C D E F G H
1 將 X X X X X X X
2 仕 X X X X X X X
3 . X X X X X X X
4 . X X X X X X X
choose position: a2
move to: a1
invalid move
  A B C D E F G H
1 將 X X X X X X X
2 仕 X X X X X X X
3 . X X X X X X X
4 . X X X X X X X
choose position: a1
move to: a2
  A B C D E F G H
1 . X X X X X X X
2 將 X X X X X X X
3 . X X X X X X X
4 . X X X X X X X
```

翻開 A1 到 A4 後仕開始往上吃，吃到將後不能吃然後將可以往下吃把仕吃掉。

1️⃣ 主程式流程圖（Main Game Loop）

```
開始
  ↓
產生棋盤 (generateChess)
  ↓
顯示棋盤 (showBoard)
  ↓
遊戲是否結束?
  ↓否
輸入棋子位置 (A2)
  ↓
棋子是否已翻開?
  ↓否
翻棋 (flip)
  ↓
重新顯示棋盤
  ↓
回到遊戲開始
```

如果棋子已翻：

```
棋子是否已翻開?
      ↓是
輸入目的位置 (B2)
      ↓
執行 move()
      ↓
是否合法移動?
      ↓否
顯示 invalid move
      ↓
重新顯示棋盤
      ↓
回到遊戲開始
```

3️⃣ 吃子規則流程圖（canEat）

```
開始
 ↓
攻擊棋子 = 兵 / 卒 ?
 ↓是
目標棋子 = 將 / 帥 ?
 ↓是 → 可以吃
 ↓否
目標棋子 = 炮 ?
 ↓是 → 不可吃
 ↓否 → 比較等級
```

如果不是兵：

```
攻擊棋子 = 將 / 帥 ?
 ↓是
目標 = 兵 / 卒 ?
 ↓是 → 不可吃
 ↓否 → 比較等級
```

最後：

```
攻擊棋子 weight ≥ 目標棋子 weight ?
 ↓是 → 可以吃
 ↓否 → 不可吃
```

2️⃣ 移動判斷流程圖（move）

```
開始 move(from,to)
      ↓
解析座標 (parse)
      ↓
來源棋子存在?
      ↓否 → return false
      ↓是
棋子是否翻開?
      ↓否 → return false
      ↓是
距離是否為 1?
      ↓否 → return false
      ↓是
目標位置是否為空?
      ↓是
移動棋子
return true
      ↓否
目標棋子是否翻開?
      ↓否 → return false
      ↓是
是否同一陣營?
      ↓是 → return false
      ↓否
是否符合吃子規則?
      ↓否 → return false
      ↓是
吃子並移動
return true
```

心得:

經過這次作業我了解到 AI 不單純只是幫我們把東西做完而是提高我們的工作效率，他們應該做的是把我們做得到的事情用高效的方式做出來，如果不會的東西就要他來教你，不是都給他做然後根本不了解他幫你做出來甚麼東西，還有程式遇到問題的時候可以跟他講我們遇到的情況，還有我覺得可能是什麼原因讓他來協助我解決問題，如果遇到不會的都可以叫他更詳細的跟我解釋或是舉例，這樣子才是真的把知識內化吸收，這就是我這次的心得

Main.java

```java
package org.example;

import java.util.Scanner;

public class Main {
```

```java
    public static void main(String[] args) {

        ChessGame game = new ChessGame();

        game.generateChess();

        Scanner sc = new Scanner(System.in);

        while (true) {

            game.showBoard();

            if (game.gameOver()) {
                System.out.println("Game Over!");
                break;
            }

            System.out.print("choose position: ");

            String pos = sc.next().toUpperCase();

            if (game.flip(pos))
                continue;

            System.out.print("move to: ");

            String dest = sc.next().toUpperCase();

            if (!game.move(pos, dest))
                System.out.println("invalid move");
        }
    }
}
```

Chess.java

```java
package org.example;
```

```java
public class Chess {

    String name;
    String side;
    int weight;
    boolean open = false;

    public Chess(String name, String side, int weight) {
        this.name = name;
        this.side = side;
        this.weight = weight;
    }

    public String toString() {

        if (!open)
            return "X";

        return name;
    }
}
```

ChessGame.java

```java
package org.example;

import java.util.*;

public class ChessGame {

    Chess[][] board = new Chess[4][8];

    public void generateChess() {

        String[] pieces = {
                "將","士","士","象","象","馬","馬","車","車",
```

```
                "炮","炮","卒","卒","卒","卒","卒",
                "帥","仕","仕","相","相","馬","馬","車","車",
                "炮","炮","兵","兵","兵","兵","兵"
        };

        List<String> list = new ArrayList<>(Arrays.asList(pieces));

        Collections.shuffle(list);

        int index = 0;

        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 8; j++) {

                String name = list.get(index);

                String side = getSide(name);

                int weight = getWeight(name);

                board[i][j] = new Chess(name, side, weight);

                index++;
            }
        }
    }

    public String getSide(String name) {

        if(name.equals("將") || name.equals("士") || name.equals("
象") ||
                name.equals("車") || name.equals("馬") ||
name.equals("炮") ||
                name.equals("卒"))
            return "black";

        return "red";
    }
```

```java
public int getWeight(String name) {

    switch (name) {

        case "將":
        case "帥":
            return 7;

        case "士":
        case "仕":
            return 6;

        case "象":
        case "相":
            return 5;

        case "車":
            return 4;

        case "馬":
            return 3;

        case "炮":
            return 2;

        case "卒":
        case "兵":
            return 1;
    }

    return 0;
}

public void showBoard() {

    System.out.println("  A B C D E F G H");
```

```java
        for (int i = 0; i < 4; i++) {

            System.out.print((i + 1) + " ");

            for (int j = 0; j < 8; j++) {

                if (board[i][j] == null)
                    System.out.print(". ");
                else
                    System.out.print(board[i][j] + " ");

            }

            System.out.println();
        }
    }

    public int[] parse(String pos) {

        if (pos.length() != 2)
            return null;

        char colChar = pos.charAt(0);
        char rowChar = pos.charAt(1);

        if (colChar < 'A' || colChar > 'H')
            return null;

        if (rowChar < '1' || rowChar > '4')
            return null;

        int col = colChar - 'A';
        int row = rowChar - '1';

        return new int[]{row, col};
    }

    public boolean flip(String pos) {
```

```java
        int[] p = parse(pos);

        if (p == null)
            return false;

        Chess c = board[p[0]][p[1]];

        if (c == null)
            return false;

        if (!c.open) {

            c.open = true;
            return true;

        }

        return false;
    }

    public boolean canEat(Chess a, Chess b) {

        String A = a.name;
        String B = b.name;

        if ((A.equals("兵") || A.equals("卒")) &&
                (B.equals("將") || B.equals("帥")))
            return true;

        if ((A.equals("將") || A.equals("帥")) &&
                (B.equals("兵") || B.equals("卒")))
            return false;

        if ((A.equals("兵") || A.equals("卒")) &&
                B.equals("炮"))
            return false;
```

```
        return a.weight >= b.weight;
    }

    public boolean move(String from, String to) {

        int[] p1 = parse(from);
        int[] p2 = parse(to);

        if (p1 == null || p2 == null)
            return false;

        Chess c1 = board[p1[0]][p1[1]];
        Chess c2 = board[p2[0]][p2[1]];

        if (c1 == null)
            return false;

        if (!c1.open)
            return false;

        int dist = Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] -
p2[1]);

        if (dist != 1)
            return false;

        if (c2 == null) {

            board[p2[0]][p2[1]] = c1;
            board[p1[0]][p1[1]] = null;

            return true;
        }

        if (!c2.open)
            return false;

        if (c1.side.equals(c2.side))
```

```
            return false;

        if (canEat(c1, c2)) {

            board[p2[0]][p2[1]] = c1;
            board[p1[0]][p1[1]] = null;

            return true;
        }

        return false;
    }

    public boolean gameOver() {

        boolean red = false;
        boolean black = false;

        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 8; j++) {

                Chess c = board[i][j];

                if (c == null)
                    continue;

                if (c.side.equals("red"))
                    red = true;

                if (c.side.equals("black"))
                    black = true;
            }
        }

        return !(red && black);
    }
}
```